Module 1 Implementation plan

Assume a typical stack:

- Frontend: Next.js/React
- API gateway / backend: Node.js (NestJS or Express + TypeScript)
- Data: PostgreSQL (core data), Redis (caching), maybe Kafka/RabbitMQ for events
- AI services: Separate Python FastAPI microservice(s) for models (ETA, congestion, recommendation)
- Maps: Leaflet or Mapbox GL on frontend + map tiles provider

M1.1 Nearby stops & live bus discovery (Member B – UX)

Goal: Given commuter location or chosen area, show nearby stops, upcoming buses, live positions, and ETA per route.

Step-by-step

1. Design core data model (DB)
    - Tables: `stops(id, name, lat, lon)`, `routes(id, name, direction)`, `route_stops(route_id, stop_id, sequence)`, `buses(id, route_id, operator_id)`.
    - Separate `bus_positions(bus_id, lat, lon, speed, heading, timestamp)` storing latest or short history.
2. Ingestion API for GPS
    - Endpoint: `POST /ingest/gps` behind API gateway.
    - Auth: API key per bus device (from "bus device registration" common workflow).
    - Validation: rate limiting, payload schema, location sanity checks.
    - Writes latest location to `bus_positions` and publishes event (`BUS_POSITION_UPDATED`) to message queue.
3. Stop discovery backend API
    - Endpoint: `GET /stops/nearby?lat=&lon=&radius=` using geospatial query (PostGIS `ST_DWithin`).
    - Returns list of stops and routes served at each stop.
4. Upcoming buses & ETAs per stop
    - Endpoint: `GET /stops/{stopId}/upcoming?limit=N`
    - Backend queries:
        - All active buses whose route passes this stop.

        - Their latest positions.
        - Calls ETA AI service (or baseline) to estimate time to this stop.
    - Response includes: bus id, route, direction, ETA (minutes), crowd level estimate.
5. Frontend map & UI
    - Implement a map view with user geolocation; show markers for stops within radius.
    - On clicking a stop, show bottom sheet listing upcoming buses with ETAs and crowd level.
    - Poll backend every X seconds or use WebSockets for push updates.
6. Performance & caching
    - Cache `GET /stops/nearby` results by rounded lat/lon for short time windows.
    - Cache `upcoming` results for a stop for 10–20 seconds to avoid hammering AI.
7. Security & error handling
    - Validate query params (lat/lon ranges).
    - Handle missing GPS data by falling back to schedule-based estimates or marking bus as "no data".

(No heavy AI here; uses outputs from ETA and crowding services.)

M1.2 AI-powered commute planner (Member A – AI & data)

Goal: User provides origin, destination, desired arrival/departure time; system suggests best route(s), buses, and departure times, considering predicted congestion and crowding.

AI framing

- Sub-problems:
    1. Pathfinding on transit network (graph problem).
    2. ETA prediction for each leg (supervised regression / time-series).
    3. Crowding estimation (supervised regression or classification).

You can start with rule-based pathfinding + learned ETA/crowding models, which is realistic and feasible.

Data needed

- Static network (similar to GTFS): stops, routes, trip schedules.
- Historical GPS traces: (bus_id, route_id, timestamp, lat, lon, speed).
- Historical realized arrival times at stops derived from GPS vs schedule.
- Historical crowd data (approximate):
    - If not available, simulate using heuristic based on time-of-day + route popularity.
    - In real deployment, from ticket validations / app check-ins.

Data preprocessing

1. Build transit graph
    - Nodes: stops.
    - Edges: route segments between consecutive stops (with route_id, direction).
    - Attributes per edge: distance, typical travel time by time-of-day, route type.
2. Prepare ETA training dataset
    - From historical trips, derive segments: `(route_id, from_stop, to_stop, departure_time, day_of_week, weather(optional)) → travel_time_seconds`.
    - Add features: hour of day, congestion index (if available), weekday/weekend.
3. Prepare crowding dataset
    - For each bus/stop/time: features like route_id, stop_idx, hour, day_of_week, historical demand; label: crowd level (0–3 or low/med/high).

Model choices

- ETA model
    - Start: Gradient Boosting Regression (XGBoost/LightGBM) – handles tabular features well.
    - Features: route_id (one-hot/embedding), segment index, hour, day_of_week, last known speed, delay vs schedule.
    - Output: travel_time for segment; chain segments for full trip ETA.
- Crowding model
    - Classification: Gradient Boosting or Random Forest (simple and powerful).
    - Output: probability for each class (low/med/high).

You don't *have* to start with deep learning; GBDT is enough for MVP.

Service architecture

1. Python AI microservice (FastAPI)
    - Endpoint 1: `POST /eta` with body `{ segments: [{ route_id, from_stop_id, to_stop_id, departure_time, context_features... }] }` → returns array of travel times.
    - Endpoint 2: `POST /crowding` with `{ route_id, stop_id, time, context... }` → returns probabilities.
    - Loads trained models from persisted files (e.g., `.pkl`).
2. Commute planner API (Node.js)
    - Endpoint: `POST /planner/commute` with `{ origin, destination, time, time_type (arrive_by/leave_after) }`.
    - Steps:
        1. Map origin/destination (lat/lon) to nearest stops.
        2. Run graph search (Dijkstra/A* with early pruning) to find candidate paths (sequence of edges).
        3. For each candidate path, call ETA service for each segment to compute end-to-end ETA.
        4. Call crowding service for key legs to estimate comfort.
        5. Compute a score combining total travel time, number of transfers, walking distance, and crowding.
        6. Return top K routes with explanation (why selected, predicted duration, crowd level).

Integration into UI

1. Frontend form for origin/destination selection (map + search) and desired time.
2. On submit, call `POST /planner/commute`; show list of options with timeline, ETAs and crowding icons.
3. Optionally allow user to "favorite" a commute (stored for alerts system).

Training pipeline

1. Offline Jupyter notebooks or Python scripts to:
    - Extract, clean, and feature-engineer data.
    - Train ETA and crowding models; evaluate MAE/accuracy.
    - Save models.
2. Simple retraining schedule (e.g., weekly batch job) later; full lifecycle will be covered under Module 3 AI-model-mgmt.

M1.3 Personalized alerts & notifications (Member C – platform/infra)

Goal: Allow commuters to subscribe to specific routes/trips/commutes and get push/email/app notifications for major delays, cancellations, or congestion spikes.

Step-by-step

1. Data model
    - `subscriptions(id, user_id, type, route_id, stop_id, commute_id, thresholds, channels)`
    - `notifications(id, user_id, type, payload, status, created_at, sent_at)`
2. Subscription management APIs
    - `POST /subscriptions` for creating a subscription (route or commute).
    - `GET /subscriptions` for listing; `DELETE /subscriptions/{id}` for removal.
    - Validation: ensure user can only manage their own subs.
3. Event sources
    - Delay events: from ETA/monitoring logic when predicted delay > threshold.
    - Cancellation events: from operator input.
    - Congestion events: from congestion prediction service.
4. Notification rule engine
    - Background worker consumes events:
        - For each event (e.g., `ROUTE_DELAY` on route X at time window Y), find matching subscriptions.
        - Apply user thresholds and dedupe (avoid spam).
        - Enqueue notifications to send.
5. Delivery channels
    - Phase 1: email (SMTP) and in-app notifications.
    - DB field `channels` (e.g., `["email","in_app"]`).
    - In-app: store notifications and show a bell icon on UI.
6. Security & privacy
    - Ensure a user's subscriptions are private.
    - Limit frequency/rate of notifications per user.
7. Monitoring
    - Track delivery success/failure, open rates (if needed), and backlog in queues.

Module 2 Implementation plan

M2.1 Citizen incident & complaint submission (Member B – UX)

Goal: Commuters submit structured incidents and complaints with rich metadata and media.

Step-by-step

1. Data model
    - `incidents(id, user_id, type, category, route_id, bus_id, lat, lon, description, status, severity, created_at, updated_at)`
    - `incident_media(id, incident_id, url, type)`
    - `complaints` table if you want separate from incidents, or a `subtype` field.
2. Submission UI
    - Mobile-friendly form with: category dropdown, route/bus selection, map for pinpointing location, description, photo/video upload.
    - Client-side validation for required fields and media size/type.
3. Backend API
    - `POST /incidents` (authenticated); saves incident and returns id.
    - Media upload either:
        - Direct to storage (e.g., S3/ImageKit) with signed URLs.
        - Or via backend `POST /incidents/{id}/media`.
4. Geo & route validation
    - Optionally check that chosen bus/route is near the reported location using latest GPS.
    - Reject obviously invalid combinations (e.g., bus not on that route).
5. Initial status & fields
    - On create: status = “Reported”, severity = null (to be set by AI in M2.2).
6. Security
    - Auth required; rate-limit incident creation per user/time.
    - Sanitize description text.

M2.2 Incident classification & routing (Member A – AI & data)

Goal: Auto-classify incident type and severity, and route to correct team (operator, zone).

AI framing

- Task 1: Text + metadata classification (category, severity).
- Task 2: Routing decision (which operator, which team).

Data needed

- Past incidents labeled by category and severity (at first can be manually labeled small dataset).
- Metadata: route, bus, time, user location, maybe crowding and congestion context.

Model choices

- Classification model
    - Start with classical: TF-IDF + Logistic Regression / Linear SVM on text + numeric features.
    - If you want more modern: small BERT-style model fine-tuned, but compute-heavier.
- Routing
    - For now: rule-based on route/zone/operator; you don't need ML here. If system grows, can model as classification too.

Pipeline

1. Feature engineering
    - Text: pre-process (lowercase, remove stopwords); build TF-IDF vectors.
    - Numeric: category chosen by user (if any), route_id, hour_of_day, location cluster.
    - Combine into feature vector.
2. Train incident classifier
    - Output 1: fine-grained category (e.g., breakdown, reckless driving, overcrowding, road block).
    - Output 2: severity level (1–3 or low/med/high).
3. Microservice
    - Endpoint: `POST /incidents/classify` with incident text + metadata; returns category probabilities and severity.
    - Deployed as a Python FastAPI service, similar to ETA service.
4. Integration with incident creation
    - After `POST /incidents`, backend sends async request/event to classification service.
    - When response arrives, update incident record with category and severity, and compute responsible team:
        - Based on route/zone/operator mapping + category (e.g., road issue → city authority, bus issue → operator).
5. Feedback loop

- Officers can override category/severity; their corrections are logged and later added into training dataset.

M2.3 Incident lifecycle tracking (Member C – workflows)

Goal: Manage incident progression through defined stages with full traceability.

Step-by-step

1. Status model
   - Allowed statuses: `Reported`, `Verified`, `In Progress`, `Resolved`, `Archived`.
   - `incident_status_history(incident_id, status, changed_by, changed_at, comment)`.
2. APIs for status transitions
   - `POST /incidents/{id}/status` with `{ new_status, comment }`.
   - Validate:
     - Only allowed roles can change from some statuses (e.g., Citizen cannot mark as Resolved).
     - Status transitions follow valid graph (e.g., cannot jump from Reported → Archived).
3. Officer & operator dashboards
   - Filter incidents by status, severity, route, time.
   - Provide quick actions (Verify, Start Work, Resolve).
4. Notifications
   - On status change, notify incident creator and relevant staff (e.g., team channel).
5. Audit & reporting
   - Use `incident_status_history` for computing resolution times, overdue detection, etc., used by AI in other modules.

M2.4 AI-based impact estimation (Member A – AI & data)

Goal: For high-severity incidents, estimate impact on routes (which routes/segments are affected, for how long), and trigger ETA updates + alerts.

AI framing

- This is more like regression + rules on top of existing ETA & congestion models.

Inputs

- Incident data: type, severity, exact location (lat/lon), route/bus, timestamp.
- Context: baseline congestion prediction for that segment from congestion model, typical recovery times for similar incidents.

Outputs

- Estimated:
  - Set of affected segments/routes.
  - Delay magnitude (extra minutes).
  - Time-to-recovery window.

Steps

1. Map incident to network entities
   - Match lat/lon to nearest road segment + matching route segments.
   - For bus incident, directly identify segment from bus's last location.
2. Historical recovery analysis
   - From past incidents: build dataset `(incident_type, severity, route_type, time_of_day, baseline_congestion) → recovery_time_minutes`.
   - Train regression model (e.g., Gradient Boosting) to estimate recovery time.
3. Delay estimation
   - For affected segments, adjust ETA model inputs to include "incident flag" and predicted recovery window, or simply add a "delay offset" derived from another regression.
4. Microservice
   - Endpoint: `POST /incidents/impact` returns `{ affected_segments, extra_delay_per_segment, recovery_time }`.
   - Called whenever a new high-severity incident is verified.
5. Integration with commute & live ETAs
   - ETA service or planning service takes impact outputs into account when computing future ETAs.
   - Alerts system (Module 1.3) uses this data to notify subscribers.
6. Continuous learning
   - After an incident is resolved, measure actual recovery vs predicted; feed back to model as new training data.

M2.5 Maintenance hotspot & priority ranking (Member C – analytics/workflows)

Goal: Aggregate historical incidents/delays to detect chronic problem segments and rank them for maintenance prioritization.

Step-by-step

1. Define aggregation granularity
    - Use “road segment + direction” as a unit (or “stop + segment”).
2. Compute metrics periodically (batch job)
    - For each segment:
        - Number of incidents by type/severity.
        - Total and average delay minutes linked to incidents.
        - Passenger impact proxy: estimated ridership through that segment.
3. Priority score formula
    - Example: `score = w1 * incidents_high_severity + w2 * total_delay + w3 * riders_affected`
    - Configurable weights stored in DB/config.
4. Ranking and storage
    - Store results in `maintenance_hotspots(segment_id, score, metrics_json, last_computed_at)`.
5. Officer & admin UI
    - Table + map view highlighting top N hotspots with filters (time range, zone).
    - Allow exporting as CSV for planning reports.
6. Integration with incident workflows
    - When planning maintenance, officers can link work orders to these hotspots and track post-fix improvement.

M2.6 Public transparency & accountability dashboard (Member B – UX)

Goal: Public, read-only dashboard showing aggregated incidents, open issues, and performance per area/operator without exposing sensitive user data.

Step-by-step

1. Data preparation layer
    - Aggregation queries or materialized views for:
        - Total incidents by category, area, operator.
        - Open vs resolved counts.
        - Average resolution time by area/operator.

2. Public API endpoints (read-only)
    - `GET /public/incidents/summary?area=&time_range=`
    - `GET /public/incidents/map?bbox=...`
    - Responses are anonymized (no user IDs, no free-text descriptions).
3. Frontend visualizations
    - Map showing incident density by area.
    - Charts: bar charts of average resolution times, stacked counts by category.
    - Table of "Top N worst areas/operators" but carefully worded.
4. Security & rate limiting
    - Open to public but with rate limits and caching.
    - Ensure internal IDs and sensitive fields are never exposed.
5. Linkage to platform brand
    - This is the page you show recruiters/demo: proves accountability and analytics.

Module 3 implementation plan (M3.1–M3.9)

M3.1 Fleet performance & SLA monitoring system

(Member B – dashboards/UX, but uses data from all services)

Goal: Show operators and transport officers key KPIs like on-time performance, average delay per route, missed trips, SLA compliance.

Steps

1. Define KPIs and SLAs clearly
    - Example KPIs: on-time arrival percentage, average delay per route, number of cancelled trips, average headway deviation.
    - SLA examples: "At least 85% trips per route must be within ±5 minutes of scheduled time", "No more than 2% cancellations per day per operator."
2. Model trips and events in the database
    - Create `trips(id, bus_id, route_id, scheduled_start_time, actual_start_time, scheduled_end_time, actual_end_time, status)` for each run of a bus.
    - Create `stop_arrivals(trip_id, stop_id, scheduled_arrival, actual_arrival)` so you can compute stop-level deviations.
3. Generate events from GPS + planner

    - When a bus passes near a stop (based on GPS and route geometry), mark an `actual_arrival` in `stop_arrivals`.
    - Compare with `scheduled_arrival` to get delay, classify as on-time/early/late.
4. Batch aggregations for KPIs
    - Nightly (or hourly) job that aggregates into `route_daily_metrics(route_id, date, on_time_pct, avg_delay, cancelled_trips, total_trips, operator_id)`.
    - Use SQL window functions / GROUP BY to compute metrics.
5. Real-time approximations
    - For dashboards, you can mix pre-computed metrics with near-real-time updates from a cache layer (e.g., Redis) updated by streaming jobs as new arrivals come in.
6. Dashboards & filters
    - Backend: `GET /analytics/fleet?route_id=&operator_id=&date_range=` returns metrics and timeseries data.
    - Frontend: interactive charts (e.g., line chart for avg delay over last 7 days, heatmap of routes vs on-time %).

M3.2 Real-time congestion monitoring & prediction system

(Member A – AI & data)

Goal: Show live congestion on a map and predict future congestion for upcoming time windows to support proactive management.

Conceptual model

- Treat the road network as segments; at each time window (e.g., 5 minutes), compute congestion level per segment from bus speeds and optionally external data (traffic sensors, weather).
- Use a time-series model (classical or deep learning) to forecast congestion per segment. Deep learning architectures that combine CNN for spatial structure and RNN/LSTM for temporal patterns are widely used for congestion and traffic prediction.

Data

- For each segment and time window:

- Average bus speed, travel time, volume estimate, number of buses.
- Time-of-day, day-of-week, holiday flag, weather features (if available).
- Label: congestion index, e.g., ratio of free-flow speed to current speed or discrete level (0–3).

Steps

1. Segment the network
    - Use the same representation as ETA module: each edge/segment with `segment_id`, geometry, and associated routes.
    - Precompute "free-flow" speed for each segment from historical off-peak data.
2. Build historical congestion dataset
    - From GPS traces, in 5-minute windows: compute actual speed per segment.
    - Compute congestion index like `congestion = free_flow_speed / current_speed` or classify into {green, yellow, red}.
    - Add contextual features (time, day, weather, events if you have them).
3. Choose model(s)
    - MVP: Gradient boosting regression/classifier (XGBoost/LightGBM) per segment or per group of segments – simple to train and often strong.
    - Ambitious version: A CNN+LSTM or graph-based model (e.g., graph convolution network) that predicts congestion across the entire network jointly, as in recent deep learning congestion forecasting work.
4. Train & evaluate
    - Split by time (train/val/test).
    - Metrics: MAE/MSE for regression or accuracy/F1 for classification.
    - Compare simple baselines (historical average) vs ML vs any deep model.
5. Deploy congestion prediction service (Python FastAPI)
    - Endpoint: `POST /congestion/predict` with list of segments and time horizon (e.g., next 30 minutes in 5-minute steps).
    - Returns predicted congestion levels and/or speeds per segment per time step.
6. Monitoring & map integration
    - Backend: `GET /congestion/current` returns latest observed congestion (aggregated).
    - `GET /congestion/forecast` calls AI service to get predictions.
    - Frontend: map layers that color segments based on observed vs predicted congestion, plus time slider to scrub future predictions.
7. Use predictions for operations
    - Feed predictions into:

- Commute planner (avoid predicted red segments).
- Alerts system (notify commuters on high congestion risk on their usual route).
- Authority dashboard recommending extra buses or reroutes.

M3.3 Route configuration & optimization system

(Member A – AI/data + admin UX)

Goal: Admins manage routes, stops, and schedules; system suggests optimizations based on demand and congestion.

Steps

1. Core CRUD for network configuration
    - Tables: `routes`, `stops`, `route_stops`, `trips_scheduled`, etc.
    - Admin UIs to create/update/delete routes, attach stops in order, and define base schedules.
2. Collect usage & performance data
    - From trips and ridership/crowding model: for each route and time band, estimate demand and load factor.
    - From congestion model: segments with chronic delays.
3. Heuristic optimisation ideas
    - If load factor consistently > threshold in certain windows → suggest adding frequency or deploying larger buses.
    - If segments show both high demand and high congestion → suggest staggered schedules, added express routes, or alternative paths.
    - If segments have low demand for long periods → suggest reducing frequency or re-routing.
4. Optimization engine (MVP)
    - Start with rule-based recommendations (if conditions > thresholds → suggestion).
    - Later: formulate optimization as a simple linear/integer programming problem such as "allocate buses to routes to maximize on-time % subject to fleet size constraints."
5. Admin interface for suggestions
    - View suggestions list with expected impact (e.g., "Adding one bus between 8–9 AM can cut average delay by X minutes").
    - Allow admin to accept and apply (updates schedules) or ignore with reason (logged).

6. Feedback loop
    - After applying changes, monitor if KPIs actually improve; feed this data back into heuristic tuning or ML models.

M3.4 Access control & role management system

(Member C – security/platform)

Goal: Enforce clean RBAC across all services (Commuter, Operator, Officer, Admin, ML/DevOps) in line with best practices like least privilege and clear role definitions.

Steps

1. Define roles and permissions matrix
    - Roles: `COMMUTER`, `OPERATOR`, `OFFICER`, `ADMIN`, `ML_DEVOPS`.
    - Permissions: e.g., `VIEW_INCIDENT`, `CHANGE_INCIDENT_STATUS`, `DEPLOY_MODEL`, `VIEW_AUDIT_LOG`, etc.
    - Document which endpoints/actions each role can perform.
2. Data model for RBAC
    - `roles(id, name)`; `permissions(id, name)`; `role_permissions(role_id, permission_id)`; `user_roles(user_id, role_id)`.
    - Optionally, hierarchical roles (Admin inherits Officer permissions).
3. Auth tokens & claims
    - During login, issue JWT containing user id and roles (or permission claims).
    - Gateway/middleware extracts roles and attaches to request context.
4. Authorization middleware
    - At API level, decorate routes with required permissions (e.g., `@RequirePermission("VIEW_ANALYTICS")`).
    - Middleware checks current user's permissions; deny and log on failure.
    - Apply same checks on WebSocket/events handlers where needed.
5. Admin UI for roles & access reviews
    - Admin screen listing users and roles; ability to assign/unassign roles with confirmation.
    - Periodic "access review" process: show stale or suspicious access (e.g., operator who hasn't logged in for months) to clean up.
6. Logging & security best practices
    - Log all role/permission changes, who did them, and from where.

- Apply principle of least privilege as default, and avoid granting Admin role casually.

M3.5 Security & audit log review system

(Member C – security & ops)

Goal: Record and review all critical actions for security and accountability.

Steps

1. Define what is "audit-worthy"
    - Examples: login attempts, role changes, device registrations, route changes, incident status changes, model deployments, config changes.
2. Audit log schema
    - `audit_logs(id, user_id, role_snapshot, action_type, resource_type, resource_id, metadata_json, ip, user_agent, created_at)`.
    - Keep metadata as JSON to record before/after states for some actions.
3. Central logging mechanism
    - Each service sends audit events (e.g., to a central logging service or message queue).
    - Central consumer writes into `audit_logs` store and also into a log index (Elasticsearch/OpenSearch) for fast search.
4. Review interface
    - Admin/Auditor UI with filters: by user, action_type, date range, resource.
    - Paginated results, detail view per event.
5. Alerting hooks
    - For specific events (e.g., multiple failed logins, suspicious role escalations), emit security alerts to the ops/infosec channel.
6. Retention & privacy
    - Define retention policy for log types; longer for security-related logs.
    - Ensure logs do not store sensitive PII unnecessarily.

M3.6 AI model training & configuration management system

(Member A – MLOps)

Goal: Manage the lifecycle of all AI models in the system (ETA, crowding, congestion, incident classification, impact estimation) in a safe, trackable way.

Steps

1. Separate "analytics/ML warehouse"
    - Periodically ETL operational data from Postgres into an analytics DB or object storage (e.g., Parquet in a data lake).
    - Keep training data separated from production OLTP database.
2. Training pipelines
    - For each model:
        - Data extraction & cleaning script.
        - Feature engineering script.
        - Model training and evaluation script (Jupyter → Python modules).
    - For ETA and bus arrival, ML literature uses models like XGBoost and deep fully-connected nets to achieve low MAE and RMSE.
    - For congestion, CNN/LSTM hybrids as mentioned earlier.
3. Model registry & metadata
    - `models(id, name, version, type, metrics_json, created_at, created_by, status)`; store serialized models in object storage.
    - Link a model version to the service that uses it (`eta_service`, `congestion_service`, etc.).
4. Deployment workflow
    - ML/DevOps user picks a model version to deploy through an admin UI.
    - System updates AI service config (e.g., environment variable or config file) and triggers a rolling restart or hot reload.
    - Log deployment as an audit event with previous and new model info.
5. Feature flags & A/B testing
    - Support flags like `eta_model=v2`, `congestion_model=v1`; allow routing small traffic percentage to a new model to compare metrics (shadow or A/B tests).
6. Monitoring model performance
    - Compare predicted vs actual travel times and congestion; compute online error metrics (MAE/RMSE) and drift indicators.
    - Expose a small dashboard for ML health.
7. Rollback capabilities
    - If metrics degrade or errors spike, ML/DevOps can quickly roll back to previous model via UI; system updates config and restarts services.

M3.7 System health & observability dashboard

(Member B – dashboards, with DevOps)

Goal: Provide a central, product-level view of system health (latency, error rates, throughput, queue backlog, resource usage).

Steps

1. Instrumentation
    - Use OpenTelemetry or similar in Node.js + Python services to expose metrics: request counts, latencies, error codes, queue sizes, DB query times.
    - Structured logging (JSON) including correlation IDs across services.
2. Metrics collection stack
    - Prometheus (or similar) scrapes metrics endpoints `/metrics`.
    - Logs shipped to a log stack (e.g., ELK or Loki).
3. Dashboards
    - Use Grafana or custom React dashboards calling an internal analytics API.
    - Panels:
        - API latency & error rate per service.
        - Queue backlog for event pipeline.
        - GPS ingestion throughput.
        - Database CPU, connections, slow queries.
4. Alerting
    - Define alerts (e.g., 5xx rate > threshold, latency > threshold, queue backlog > threshold).
    - Send alerts to DevOps/ML channels and optionally display in admin UI.
5. Admin UI integration
    - Simple in-product dashboard summarizing high-level status (green/yellow/red) for non-technical admins, pulling from underlying metrics.

M3.8 Data export & integration API system

(Member B – API & UX)

Goal: Allow external tools (researchers, planners, other gov systems) to access aggregated, anonymized transit data via exports and APIs.

Steps

1. Define allowed datasets
    - Aggregated route performance, incident statistics, congestion heatmaps.
    - Never expose raw PII or fine-grained traces that can re-identify commuters.
2. Export APIs
    - `GET /api/export/routes/performance?date_range=&format=csv`
    - `GET /api/export/incidents/summary?area=&time_range=`
    - Implement streaming/ pagination for large datasets.
3. Integration APIs
    - REST endpoints for programmatic access to metrics (e.g., for city planning tools).
    - Scoped API keys or OAuth clients for trusted partners; enforce quotas and rate limits.
4. Admin management UI
    - Admins can issue/revoke API keys, set scopes (which datasets allowed), and view usage logs.
5. Security & compliance
    - Make sure exports adhere to privacy and data-sharing policies.
    - Log all exports and API access in audit logs.

M3.9 Backup, retention & disaster recovery management system

(Member C – infra)

Goal: Ensure critical data (routes, trips, incidents, logs, models) is safely backed up, retained appropriately, and restorable after failures.

Steps

1. Classify data & set retention
    - Operational DB (Postgres): trips, stops, routes, incidents.
    - Object storage: media, model files, logs.
    - Decide retention per type (e.g., raw logs 90 days, audit logs 1–3 years, critical operational data long-term).
2. Automated backups
    - Daily full DB backups with point-in-time recovery if possible.
    - Versioned buckets for object storage with lifecycle rules (transition to cheaper tiers).
3. Backup configuration & monitoring UI

    - Admin can see when last backup succeeded, how much storage is used, and define retention policies for each category.
    - Show status indicators for each backup job.
4. Restore and DR drills
    - Provide internal tools or scripts to restore backups to a staging environment.
    - Periodic DR tests: simulate failure, restore to known point, validate consistency.
5. Geo-redundancy (if budget allows)
    - Replicate backups to a second region for resilience against regional failures.
6. Documentation & runbooks
    - DR runbook: step-by-step instructions for admins on how to respond to outages or data loss events.